

Nama :Aliq Nur Shiddiq

NIM :607012400101

Kelas :4804

TUGAS PENDAHULUAN MODUL 5

Form dan Input Edit Delete Data

1. Apa perbedaan antara metode POST dan GET dalam pengiriman data melalui form HTML? Berikan contoh kapan lebih aman menggunakan POST daripada GET.

Jawab: Perbedaan dari aspek pengiriman data, GET data dikirim lewat URL (contoh: example.com?nama=Ali). Kalau POST Data dikirim lewat body, tidak terlihat di URL.

Dari segi keamanan juga untuk GET kurang aman, karena data bisa dilihat di URL dan tersimpan di Riwayat browser. Dan POST lebih aman, karena data tersembunyi dari URL.

```
<form method="POST" action="/login">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>
```

2. Jelaskan pentingnya validasi data input dalam PHP. Apa yang dapat terjadi jika data input tidak divalidasi?

Jawab: **Validasi data** adalah proses memeriksa apakah data yang dimasukkan oleh user sudah sesuai dengan yang diharapkan (misalnya tidak kosong, format benar, tidak mengandung karakter berbahaya, dll). Jika data input tidak divalidasi akan terjadi Error program atau crash, Masuknya data sampah ke database, dan celah keamanan.

3. Mengapa penting menggunakan atribut required dalam form input HTML? Jelaskan dengan contoh sederhana.

Jawab: Atribut required digunakan untuk **memastikan input wajib diisi** sebelum form dikirim. Jika field yang ditandai required masih kosong, maka browser akan **menolak submit form** dan menampilkan pesan peringatan.

```
<form action="proses.php" method="post">
  <label>Nama:</label>
  <input type="text" name="nama" required>
  <button type="submit">Kirim</button>
</form>
```

Jika user mencoba untuk mengirim form tanpa mengisi nama, maka akan muncul pesan otomatis seperti: "Please fill out this field."

4. Mengapa metode POST lebih aman daripada GET saat melakukan penghapusan data? Jelaskan risiko yang mungkin terjadi jika GET digunakan untuk operasi penghapusan.

Jawab: Penghapusan data menggunakan POST adalah metode untuk menghapus data dari server dengan mengirimkan permintaan HTTP POST. Ini lebih aman dibandingkan metode GET karena tidak menampilkan data di URL dan memungkinkan pengiriman data yang lebih kompleks. Biasanya disertai konfirmasi dari pengguna sebelum proses dilakukan.

Risiko jika menggunakan GET untuk operasi penghapusan yaitu URL bisa diakses langsung, Link bisa disimpan di bookmark atau history, dan **Bisa dipicu oleh bot atau crawler.**

5. Sebutkan langkah-langkah dasar dalam menyimpan data dari form HTML ke dalam database MySQL menggunakan MySQLi.

- Buat form HTML

```
<form method="POST" action="simpan.php">
  <input type="text" name="nama" placeholder="Nama" required>
  <input type="email" name="email" placeholder="Email" required>
  <button type="submit">Simpan</button>
</form>
```

- Buat file PHP untuk menyimpan data (simpan.php)

```
<?php
// Langkah 1: Koneksi ke database
$koneksi = mysqli_connect("localhost", "root", "", "nama_database");

// Langkah 2: Ambil data dari form
$nama = $_POST['nama'];
$email = $_POST['email'];

// (Opsional) Validasi data sederhana
if (empty($nama) || empty($email)) {
    echo "Data tidak boleh kosong!";
    exit;
}

// Langkah 3: Simpan data ke tabel
$sql = "INSERT INTO users (nama, email) VALUES ('$nama', '$email')";
$query = mysqli_query($koneksi, $sql);

// Langkah 4: Cek apakah berhasil
if ($query) {
    echo "Data berhasil disimpan!";
} else {
    echo "Gagal menyimpan data: " . mysqli_error($koneksi);
}
?>
```

6. Buat sebuah script PHP sederhana untuk menampilkan form edit data berdasarkan Primary Key yang dikirim melalui URL menggunakan metode GET. Jelaskan proses yang terjadi dalam script tersebut.

Jawab:

- File: edit.php

```
1 <?php
2 // Langkah 1: Koneksi ke database
3 $koneksi = mysqli_connect("localhost", "root", "", "nama_database");
4
5 // Langkah 2: Ambil ID dari URL (GET)
6 $id = $_GET['id'];
7
8 // Langkah 3: Ambil data berdasarkan ID
9 $sql = "SELECT * FROM users WHERE id = $id";
10 $query = mysqli_query($koneksi, $sql);
11 $data = mysqli_fetch_assoc($query);
12
13 // Langkah 4: Tampilkan form dengan data lama
14 ?>
15 <form action="update.php" method="POST">
16   <input type="hidden" name="id" value="<?= $data['id'] ?>">
17   <label>Nama:</label>
18   <input type="text" name="nama" value="<?= $data['nama'] ?>" required><br>
19   <label>Email:</label>
20   <input type="email" name="email" value="<?= $data['email'] ?>" required><br>
21   <button type="submit">Update</button>
22 </form>
```

- Proses yang terjadi:
 - User mengakses URL: edit.php?=3, artinya kita mau edit data user dengan id=3.
 - Ambil ID dari URL: \$_GET['id'] mengambil nilai id dari parameter di URL.
 - Query ke database: Script mencari data user di tabel users berdasarkan id.
 - Data ditampilkan ke form: Nilai nama dan email akan otomatis terisi (pre-filled) di dalam form.
 - Saat disubmit: Form akan mengirim data ke update.php dengan method POST untuk diproses (diperbarui).

7. Jelaskan perbedaan antara mysqli_error (procedural) dan mysqli->error (object-oriented) dalam menangani kesalahan pada query MySQLi. Kapan sebaiknya menggunakan masing-masing metode tersebut?

- mysqli_error() (Procedural)
 - Menggunakan gaya procedural
 - Perlu menyertakan parameter koneksi
 - Cocok digunakan untuk script yang sederhana atau non-OOP
- \$mysqli->error (Object-Oriented)
 - Menggunakan gaya object-oriented (OOP)
 - Tidak perlu parameter, langsung mengakses property dan objek koneksi

- Lebih cocok digunakan pada aplikasi berbasis OOP atau framework

Situasi yang sebaiknya digunakan yaitu, gunakan `mysqli_error()` saat script procedural biasa atau sederhana. Gunakan `$mysqli->error` saat menggunakan aplikasi berbasis OOP atau pakai framework.

8. Buat contoh kode PHP untuk menangani konfirmasi perubahan data yang dilakukan oleh pengguna, sebelum perubahan data disimpan ke database.

- File: `edit.php` – Form Edit

```
// Koneksi database
$koneksi = mysqli_connect("localhost", "root", "", "dbku");

// Ambil data berdasarkan ID
$id = $_GET['id'];
$data = mysqli_fetch_assoc(mysqli_query($koneksi, "SELECT * FROM users WHERE id = $id"));
?>

<form action="konfirmasi.php" method="POST">
  <input type="hidden" name="id" value="<?= $data['id'] ?>">
  Nama: <input type="text" name="nama" value="<?= $data['nama'] ?>"><br>
  Email: <input type="email" name="email" value="<?= $data['email'] ?>"><br>
  <button type="submit">Lanjutkan</button>
</form>
```

- File: `konfirmasi.php` – Tampilkan Informasi

```
<?php
// Tangkap data dari edit.php
$id = $_POST['id'];
$nama = $_POST['nama'];
$email = $_POST['email'];
?>

<h3>Apakah Anda yakin ingin mengubah data berikut?</h3>
<p>Nama: <?= $nama ?></p>
<p>Email: <?= $email ?></p>

<form action="simpan_update.php" method="POST">
  <input type="hidden" name="id" value="<?= $id ?>">
  <input type="hidden" name="nama" value="<?= $nama ?>">
  <input type="hidden" name="email" value="<?= $email ?>">
  <button type="submit">Ya, Simpan Perubahan</button>
</form>

<a href="edit.php?id=<?= $id ?>">Batal</a>
```

- File: simpan_update.php – Simpan Perubahan

```
<?php
$koneksi = mysqli_connect("localhost", "root", "", "dbku");

$id = $_POST['id'];
$nama = $_POST['nama'];
$email = $_POST['email'];

$sql = "UPDATE users SET nama='$nama', email='$email' WHERE id=$id";
if (mysqli_query($koneksi, $sql)) {
    echo "Data berhasil diperbarui!";
} else {
    echo "Gagal: " . mysqli_error($koneksi);
}
```

- Penjelasan Singkat:
 - Data diedit → dikirim ke konfirmasi.php → user diberi kesempatan mengecek ulang.
 - Jika setuju, data dikirim ke simpan_update.php untuk disimpan ke database.
 - Ini memberi **kontrol ekstra** agar pengguna yakin sebelum perubahan dilakukan
9. Apa yang dimaksud dengan proses validasi data pada form input? Berikan contoh cara validasi sederhana pada data nama dan email menggunakan PHP.
- Validasi data adalah proses untuk memastikan bahwa data yang dimasukkan oleh pengguna ke dalam form sudah benar, lengkap, dan sesuai format sebelum diproses lebih lanjut (misalnya disimpan ke database).
 - Mencegah kesalahan input (misalnya field kosong atau salah format).
 - Menghindari data tidak valid masuk ke database.
 - Menjaga keamanan dari serangan seperti SQL Injection atau XSS

- Contoh Validasi Sederhana: Nama dan Email dengan PHP

```
//Nomor 9
$nama = $_POST['nama'];
$email = $_POST['email'];
$errors = [];

// Validasi nama
if (empty($nama)) {
    $errors[] = "Nama tidak boleh kosong.";
}

// Validasi email
if (empty($email)) {
    $errors[] = "Email tidak boleh kosong.";
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = "Format email tidak valid.";
}

// Tampilkan hasil validasi
if (!empty($errors)) {
    foreach ($errors as $error) {
        echo "<p style='color:red;'>$error</p>";
    }
} else {
    echo "Data valid. Siap diproses.";
}
?>
```

- Penjelasan Singkat:
 - empty() digunakan untuk mengecek apakah input kosong.
 - filter_var() dengan FILTER_VALIDATE_EMAIL digunakan untuk memastikan email sesuai format.
 - Jika ada error, ditampilkan ke user; jika tidak, lanjut proses.

10. Buat sebuah script sederhana yang memungkinkan pengguna untuk menghapus data berdasarkan URL parameter (misalnya, delete.php?id=3). Jelaskan bagaimana Anda memastikan bahwa hanya data yang diminta yang akan dihapus.

- Script PHP: Hapus Data Berdasarkan Parameter URL (delete.php?id=3)

```
// Koneksi ke database
$koneksi = mysqli_connect("localhost", "root", "", "dbku");

// Cek apakah parameter ID dikirim melalui URL
if (isset($_GET['id'])) {
    $id = (int) $_GET['id']; // Konversi ke integer untuk keamanan

    // Query hapus data berdasarkan ID
    $query = "DELETE FROM users WHERE id = $id";
    $hapus = mysqli_query($koneksi, $query);

    // Cek apakah penghapusan berhasil
    if ($hapus) {
        echo "Data dengan ID $id berhasil dihapus.";
    } else {
        echo "Gagal menghapus data: " . mysqli_error($koneksi);
    }
} else {
    echo "ID tidak ditemukan di URL.";
}
```

- Penjelasan script
 - `$_GET['id']` mengambil nilai ID dari URL.
 - `(int)` memastikan hanya angka yang bisa digunakan (menghindari SQL injection dasar).
 - Query `DELETE FROM users WHERE id = $id` hanya akan menghapus data berdasarkan ID yang dikirim.
 - `mysqli_query` menjalankan perintah hapus.
 - Jika berhasil, pesan sukses ditampilkan.
- Bagaimana Memastikan Hanya Data yang Diminta yang Terhapus?
 - Gunakan konversi ke int: `(int) $_GET['id']` memastikan nilai ID adalah angka valid.
 - Gunakan kondisi `WHERE id = ...`: membatasi penghapusan hanya pada 1 baris yang cocok.